Universidad del Rosario

Algoritmos y Estructura de Datos ·

Laboratorio integrador — Corte 3

Árboles y grafos 60 minutos · 100 puntos

Laboratorio práctico en computador, en parejas, con verificación automática.

Estudiante 1	Estudiante 2	Fecha
Maria Fernanda Morales Bernal		09 / 11 / 2026

Duración	Nivel	Modalidad	Puntaje
60 minutos estrictos	Intermedio	Parejas, en computador	100 puntos

Objetivo

Integrar en una sola sesión las cuatro estructuras del Corte 3: el árbol binario de búsqueda como índice, el árbol AVL como índice que no se degrada, el recorrido en anchura para rutas mínimas y el recorrido en profundidad para conectividad. Los cuatro retos se aplican sobre el mismo sistema, RedAcopio, de modo que el laboratorio no es una colección de ejercicios sueltos sino un módulo completo.

Qué se evalúa en cada reto

Reto	Tema	Lenguaje	Pruebas	Mesa	Total
1	Árbol binario de búsqueda	Python	20	5	25
2	Rotaciones y AVL	C++	20	5	25
3	Grafos y BFS	Python	20	5	25
4	DFS, componentes y vías críticas	C++	20	5	25
		TOTAL	80	20	100

«Pruebas» son los puntos que otorga el script de verificación. «Mesa» son los 5 puntos de la pregunta que la profesora hace en el puesto sobre el código que ustedes escribieron: la responde cualquiera de los dos, elegido al azar.

.

Instrucciones generales

- Trabajo en parejas, en computador. Las dos personas deben poder explicar cualquier línea del código entregado.
- Permitido: sus apuntes, el repositorio del curso y la documentación oficial de C++ y de Python. No permitido: código de otras parejas ni asistentes de inteligencia artificial.
- Modifique únicamente indice.py, balanceo.cpp, rutas.py y conectividad.cpp. Los archivos pruebas_*, avl.h y red.h no se pueden modificar: alterarlos se considera fraude académico según la sección 13 de la guía.
- Ejecute las pruebas con bash verificar.sh en Linux o macOS, o con verificar.bat en Windows. Para un solo reto, agregue su nombre: bash verificar.sh R3.
- Cuando un reto queda resuelto, las pruebas imprimen su código de cierre. Ese código no está escrito en ningún archivo: se calcula a partir de los resultados correctos. Cópielo en la hoja de respuestas.
- Antes de empezar, ejecute verificar.sh una vez: debe imprimir los cuatro retos en ABIERTO con sus listas de FALLA. Si algo distinto ocurre, avise de inmediato: es un problema de entorno, no suyo.

La red del laboratorio

Los retos 3 y 4 trabajan sobre esta red de RedAcopio: diez puntos y once vías. Está definida en rutas.py y en red.h; no la modifique.

La red · 10 vértices, 11 aristas

```
NOMBRES = ["ACOPIO","NORTE","SUR","ORIENTE","OCCIDENTE",
           "PLANTA","TALLER","PUERTO","VEREDA","MINA"]
ARISTAS = [(0,1),(0,2),(0,3),(1,4),(1,5),(2,7),(3,6),(5,6),(4,7),(7,8),(6,9)]
```



RETO 1 · Árbol binario de búsqueda · Python · 25 puntos

Archivo que debe modificar: `R1/indice.py` · Pruebas: `python3 pruebas_r1.py`

Sistema RedAcopio — índice de recursos por código. El archivo tiene tres funciones por implementar y dos ya resueltas que puede usar como referencia.

Lo que debe implementar

Función	Qué debe hacer	Puntos
insertar(n, codigo, nombre)	Inserta respetando la invariante del BST y devuelve la raíz del subárbol. Si el código ya existe, no se inserta. Use el patrón de reenganche: n.izq = insertar(n.izq, …)	8
altura(n)	Número de aristas del camino más largo hasta una hoja. Árbol vacío: −1. Un solo nodo: 0. Recursiva.	6
buscar(n, codigo)	Devuelve la tupla (nodo, comparaciones). Si el código no está, el nodo es None. Debe bajar comparando, no recorrer todo el árbol.	6

inorden y minimo ya están implementados.

Datos con los que se prueba

Se insertan en ese orden

```
DATOS = [(450,"Taladro"), (220,"Proyector"), (780,"Carpa"), (130,"Megafono"),
         (310,"Mesa"),    (640,"Bascula"),   (890,"Carretilla"), (275,"Extension")]
```

Nueve pruebas verifican el recorrido ordenado, la altura en tres casos (árbol de ocho nodos, de un nodo y vacío), el mínimo, dos búsquedas que existen, una que no, y que un código repetido no se inserte.

Punto de atención. Si olvida la asignación del reenganche, el nodo se crea y se pierde: el árbol no cambia y el programa no da ningún error. Es el defecto silencioso de la sesión 36.

RETO 2 · Rotaciones y AVL · C++ · 25 puntos

Archivo que debe modificar: `R2/balanceo.cpp`

Pruebas: `g++ -std=c++17 -g -o pruebas_r2 balanceo.cpp pruebas_r2.cpp && ./pruebas_r2`

Sistema RedAcopio — el índice se degrada cuando los códigos llegan consecutivos. El archivo trae rotarDerecha resuelta y dos de los cuatro casos de desbalanceo ya escritos.

Lo que debe implementar

Parte	Qué debe hacer	Puntos
rotarIzquierda(x)	Es la simétrica de rotarDerecha, que ya está escrita. Tres punteros y dos recalc. Cuide el orden: primero el que baja, luego el que sube.	10
Caso RR en insertar	Factor de equilibrio menor que −1 y la clave entró a la derecha del hijo derecho. Una sola rotación.	5
Caso LR en insertar	Factor de equilibrio mayor que 1 y la clave entró a la derecha del hijo izquierdo. Rotación sobre el hijo y luego sobre la raíz.	5

Los casos LL y RL ya están escritos: úselos como plantilla. Las funciones alt, fe, recalc y esAVL también están listas.

Qué verifican las catorce pruebas

- rotarIzquierda aislada sobre el árbol 10 → 20 → 30: la raíz debe quedar en 20 y la altura en 1.
- Caso RR: insertar 100 a 700 en orden ascendente debe dar altura 2, raíz 400 y condición AVL cumplida.
- Caso LR: insertar 300, 100, 200 debe dejar la raíz en 200.
- Los casos LL y RL, que ya funcionan, no deben romperse al agregar los suyos.
- Orden descendente: siete claves de 700 a 100 deben dar también altura 2.

Punto de atención. Si recalcula primero la altura del nodo que sube, se computa con la altura vieja del que baja y queda mal. El árbol parece correcto pero los factores de equilibrio se desajustan en silencio.

RETO 3 · Grafos y recorrido en anchura · Python · 25 puntos

Archivo que debe modificar: `R3/rutas.py` · Pruebas: `python3 pruebas_r3.py`

Sistema RedAcopio — rutas mínimas entre puntos de la red. La función camino ya está implementada y reconstruye la ruta a partir del arreglo padre.

Lo que debe implementar

Función	Qué debe hacer	Puntos
construir()	Devuelve la lista de adyacencia: N listas. El grafo es no dirigido, así que cada arista se registra en los dos extremos. Cada lista de vecinos queda ordenada de menor a mayor.	6
bfs(inicio)	Devuelve (orden, dist, padre). orden es la secuencia en que se atendieron los vértices; dist[v] los tramos desde el inicio; padre[v] el vértice desde el que se llegó a v, y −1 para el inicio.	10
por_niveles(dist)	Devuelve una lista de listas: en la posición d, los NOMBRES de los vértices que están a distancia d.	4

Resultados que deben salir

Diez pruebas verifican esto y la lista de adyacencia

```
distancias desde ACOPIO : [0, 1, 1, 1, 2, 2, 2, 2, 3, 3]
niveles                 : ACOPIO | NORTE SUR ORIENTE | OCCIDENTE PLANTA TALLER PUERTO | VEREDA MINA
ruta ACOPIO -> VEREDA   : ACOPIO -> SUR -> PUERTO -> VEREDA   (3 tramos)
ruta ACOPIO -> MINA     : ACOPIO -> ORIENTE -> TALLER -> MINA (3 tramos)
```

Punto de atención. El vértice se marca como visitado al ENCOLARLO, no al sacarlo de la cola. Si lo marca al sacarlo, un mismo vértice entra varias veces y las distancias salen mal. Use `collections.deque` y `popleft()`: con `lista.pop(0)` el BFS deja de ser O(V+E).

RETO 4 · DFS, componentes y vías críticas · C++ · 25 puntos

Archivo que debe modificar: `R4/conectividad.cpp`

Pruebas: `g++ -std=c++17 -g -o pruebas_r4 conectividad.cpp pruebas_r4.cpp && ./pruebas_r4`

Sistema RedAcopio — qué vías no pueden cerrarse. La función construir, los nombres y las aristas están en red.h y no se modifican.

Lo que debe implementar

Función	Qué debe hacer	Puntos
cuantasComponentes(g)	Cuenta en cuántos pedazos está partida la red. Lance un DFS desde cada vértice no visitado: cada DFS que arranca es una componente nueva. Use una pila explícita (vector con push_back y pop_back), no recursión.	12
viasCriticas()	Para cada arista, arme la red SIN esa arista y cuente componentes. Si pasa de 1 a más de 1, la vía es crítica. Devuelva los nombres como "ORIGEN-DESTINO", en el orden en que aparecen en ARISTAS.	8

Qué verifican las siete pruebas

Caso	Resultado esperado
La red completa	1 componente
Red sin ninguna arista	10 componentes: cada punto aislado
Solo la arista ACOPIO–NORTE	9 componentes
Tres aristas sueltas	7 componentes
Cerrar PUERTO–VEREDA	2 componentes: la red se parte
Cerrar ACOPIO–NORTE	1 componente: hay rutas alternas
Lista de vías críticas	Dos vías, en el orden de ARISTAS

Punto de atención. Sin el arreglo de visitados, el DFS da vueltas para siempre: la red tiene ciclos. Y el orden de la lista que devuelve viasCriticas importa: debe seguir el orden de ARISTAS, no el alfabético.

Universidad del Rosario

HOJA DE RESPUESTAS — entregar firmada

Diligencie esta página y entréguela a la profesora al terminar.

Estudiante 1	Código
Maria Fernanda Morales Bernal	

Estudiante 2	Código

1. Códigos de cierre

Transcriba exactamente lo que imprimieron las pruebas. Si un reto quedó abierto, escriba ABIERTO y cuántas pruebas quedaron en OK.

Reto	Tema	Código de cierre	Pruebas en OK
1	BST (Python)	3301-383130	9 de 9
2	AVL (C++)	3302-24002002	14 de 14
3	BFS (Python)	3303- 31744	10 de 10
4	DFS (C++)	3304- 291011	7 de 7

Reto 1

Código:

```
# ============================================================
#  Laboratorio integrador · Corte 3 · RETO 1  (25 puntos)
#  Sesiones 35 y 36 — Arboles binarios y BST
#  Sistema: RedAcopio — indice de recursos por codigo
#
#  MODIFIQUE SOLO ESTE ARCHIVO.  NO TOQUE pruebas_r1.py
#  Ejecute:   python3 pruebas_r1.py
# ============================================================

class Nodo:
    def __init__(self, codigo, nombre):
        self.codigo = codigo
        self.nombre = nombre
        self.izq = None
        self.der = None

def insertar(n, codigo, nombre):
```

```
    """Inserta en el BST respetando la invariante: menores a la izquierda, mayores a la derecha. Si el codigo ya existe, NO se inserta. Devuelve la raiz del subarbol (patron de reenganche)."""

    if n is None:
        return Nodo(codigo, nombre)         # Se llega a un hueco entonces se crea el nodo y se devuelve
    if codigo < n.codigo:
        n.izq = insertar(n.izq, codigo, nombre)   # En este caso el recorrido es por la izquierda, se reengancha el subarbol izquierdo
    elif codigo > n.codigo:
        n.der = insertar(n.der, codigo, nombre)
    # Cuando el codigo es igual, no se hace nada es decir que no se inserta como en los otros casos
    return n

def inorden(n, salida):
    """Ya implementado: recorre izquierda, raiz, derecha."""

    if n is None:
        return
    inorden(n.izq, salida)
    salida.append(n.codigo)
    inorden(n.der, salida)

def altura(n):
    """Numero de aristas del camino mas largo de n hasta una hoja. Un arbol vacio tiene altura -1; un solo nodo, altura 0."""

    if n is None: return -1 #  Arbol vacio
    return 1 + max(altura(n.izq), altura(n.der)) # Se suma 1 por la arista que conecta el nodo actual con su hijo y se toma el máximo entre las alturas de los subárboles izquierdo y derecho.

def buscar(n, codigo):
    """Devuelve (nodo, comparaciones). Si no esta, el nodo es None.Debe bajar comparando, NO recorrer todo el arbol."""

    comparaciones = 0
    while n is not None:                # Aqui es donde se hace la busqueda, mientras n no sea None, se compara el codigo con el codigo del nodo actual
        comparaciones += 1
        if codigo == n.codigo: return n, comparaciones
        n = n.izq if codigo < n.codigo else n.der
```

```
    return None, comparaciones          # Si se sale del while, significa que no se encontro el nodo, entonces se devuelve None y el numero de comparaciones realizadas

def minimo(n):
    """Ya implementado: el extremo izquierdo."""
    while n.izq is not None:
        n = n.izq
    return n
```

ejecución:

```
RETO 1 · indice de recursos (BST) · verificacion
  [OK ] inorden queda ordenado: obtuvo [130, 220, 275, 310, 450, 640, 780, 890]
  [OK ] altura del arbol: obtuvo 3
  [OK ] altura de un solo nodo: obtuvo 0
  [OK ] altura del arbol vacio: obtuvo -1
  [OK ] minimo: obtuvo 130
  [OK ] buscar 310 (existe): obtuvo ('Mesa', 3)
  [OK ] buscar 890 (el mas profundo a la derecha): obtuvo ('Carretilla', 3)
  [OK ] buscar 500 (no existe): obtuvo None
  [OK ] codigo repetido no se inserta (siguen 8 nodos): obtuvo 8

  RETO 1 CERRADO — codigo de cierre: 3301-383130
PS C:\Users\prestamour\Downloads\Git\Maria-Fernanda-Morales\Maria-Fernanda-Morales>
```

reto 2

código:

```
// ============================================================
//  Laboratorio integrador · Corte 3 · RETO 2  (25 puntos)
//  Sesiones 37 y 38 — Rotaciones y arboles AVL
//  Sistema: RedAcopio — el indice se degrada con codigos consecutivos
//
//  Este archivo NO tiene main(): el main esta en pruebas_r2.cpp
//  MODIFIQUE SOLO ESTE ARCHIVO.  NO TOQUE pruebas_r2.cpp
//  Compile y ejecute:
//     g++ -std=c++17 -g -o pruebas_r2 balanceo.cpp pruebas_r2.cpp
//     ./pruebas_r2
// ============================================================
#include <algorithm>
#include "avl.h"
using namespace std;
```

```
int  alt(Nodo* n)    { return n == nullptr ? -1 : n->alt; }
int  fe(Nodo* n)     { return n == nullptr ? 0 : alt(n->izq) - alt(n->der); }
void recalc(Nodo* n) { n->alt = 1 + max(alt(n->izq), alt(n->der)); }

Nodo* rotarDerecha(Nodo* y) {
   //      y            x
   //     / \          / \
   //    x   C   ==>    A   y
   //   / \              / \
   //  A   B            B   C
   Nodo* x = y->izq;
   Nodo* B = x->der;
   x->der = y;
   y->izq = B;
   recalc(y); recalc(x);     // primero el que BAJA, luego el que SUBE
   return x;
}

Nodo* rotarIzquierda(Nodo* x) {
   //    x              y
   //   / \            / \
   //  A   y     ==>    x   C
   //     / \          / \
   //    B   C        A   B
   //
   // PENDIENTE: implementar. Es la simetrica de rotarDerecha.

   Nodo* y = x->der;
   Nodo* B = y->izq;
   y->izq = x;
   x->der = B;
   recalc(x); recalc(y);     // primero el que BAJA, luego el que SUBE
   return y;
}

Nodo* insertar(Nodo* n, int clave) {
   // FASE 1 — insercion normal de BST
   if (n == nullptr) return new Nodo(clave);
```

```
    if     (clave < n->clave) n->izq = insertar(n->izq, clave);
    else if (clave > n->clave) n->der = insertar(n->der, clave);
    else return n;

    // FASE 2 — actualizar la altura de este nodo
    recalc(n);

    // FASE 3 — revisar el equilibrio y rotar si hace falta
    int b = fe(n);

    if (b > 1 && clave < n->izq->clave) {          // caso LL  (ya resuelto)
       return rotarDerecha(n);
    }
    // PENDIENTE: caso RR   -> fe < -1 y la clave entro a la DERECHA del hijo derecho

    if (b < -1 && clave > n->der->clave) {         // Este es el caso RR
       return rotarIzquierda(n);
    }

    // PENDIENTE: caso LR   -> fe >  1 y la clave entro a la DERECHA del hijo izquierdo

    if (b > 1 && clave > n->izq->clave) {          // Este es el caso LR
       n->izq = rotarIzquierda(n->izq);
       return rotarDerecha(n);
    }

    // PENDIENTE: caso RL   -> fe < -1 y la clave entro a la IZQUIERDA del hijo derecho

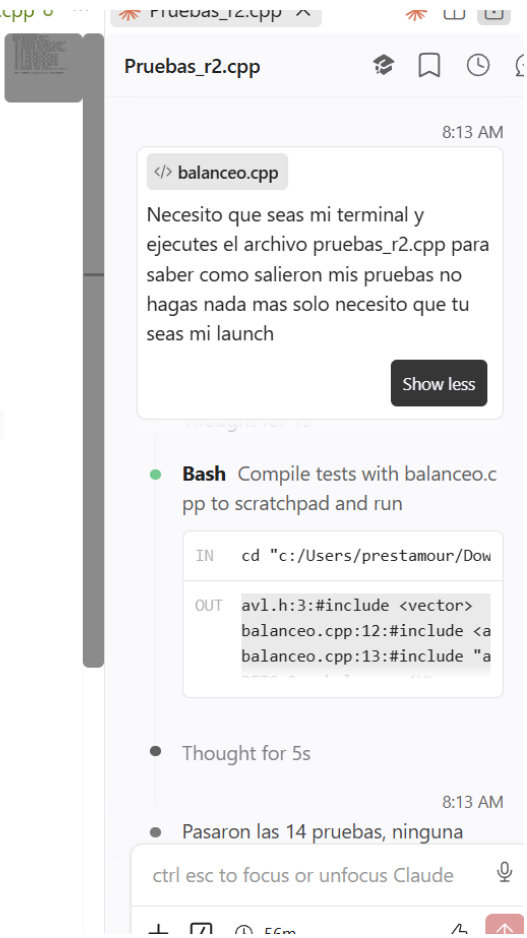
    if (b < -1 && clave < n->der->clave) {         // Este es el caso RL
       n->der = rotarDerecha(n->der);
       return rotarIzquierda(n);
    }
    return n;
}

bool esAVL(Nodo* n) {
    if (n == nullptr) return true;
    return abs(fe(n)) <= 1 && esAVL(n->izq) && esAVL(n->der);
}
```

Ejecución

```
avl.h:3:#include <vector>
balanceo.cpp:12:#include <algorithm>
balanceo.cpp:13:#include "avl.h"
RETO 2 · balanceo AVL · verificacion
  [OK ] rotarIzquierda deja de raiz el 20: obtuvo 20
  [OK ] rotarIzquierda: hijo izquierdo: obtuvo 10
  [OK ] rotarIzquierda: hijo derecho: obtuvo 30
  [OK ] rotarIzquierda: altura resultante: obtuvo 1
  [OK ] RR: altura con 7 claves ascendentes: obtuvo 2
  [OK ] RR: sigue ordenado: obtuvo 1
  [OK ] RR: la raiz quedo en 400: obtuvo 400
  [OK ] RR: cumple la condicion AVL: obtuvo 1
  [OK ] LR: la raiz quedo en 200: obtuvo 200
  [OK ] LR: cumple la condicion AVL: obtuvo 1
  [OK ] LL: la raiz quedo en 200: obtuvo 200
  [OK ] RL: la raiz quedo en 200: obtuvo 200
  [OK ] descendente: altura: obtuvo 2
  [OK ] descendente: cumple la condicion AVL: obtuvo 1

  RETO 2 CERRADO — codigo de cierre: 3302-24002002
```



Se ejecuto con Claude code como terminal porque no tiene compilador de c++

Reto 3

Código:

```
# ============================================================
#  Laboratorio integrador · Corte 3 · RETO 3  (25 puntos)
#  Sesiones 40 y 42 — Representacion de grafos y BFS
#  Sistema: RedAcopio — rutas minimas entre puntos de la red
#
#  MODIFIQUE SOLO ESTE ARCHIVO.  NO TOQUE pruebas_r3.py
#  Ejecute:   python3 pruebas_r3.py
# ============================================================
from collections import deque
```

```
NOMBRES = ["ACOPIO","NORTE","SUR","ORIENTE","OCCIDENTE",
        "PLANTA","TALLER","PUERTO","VEREDA","MINA"]
ARISTAS = [(0,1),(0,2),(0,3),(1,4),(1,5),(2,7),(3,6),(5,6),(4,7),(7,8),(6,9)]
N = len(NOMBRES)

def construir():
    """Devuelve la lista de adyacencia: una lista de N listas.
       El grafo es NO DIRIGIDO, asi que cada arista se registra en los dos
       extremos. Cada lista de vecinos debe quedar ORDENADA de menor a mayor.

       PENDIENTE: implementar."""
    adj = [[] for _ in range(N)]
    for u, v in ARISTAS:
        adj[u].append(v)
        adj[v].append(u)
    for vecinos in adj:
        vecinos.sort()
    return adj

def bfs(inicio):
    """Recorrido en anchura desde 'inicio'.
       Devuelve la tupla (orden, dist, padre):
         orden  lista de vertices en el orden en que se atendieron
         dist   dist[v] = numero de tramos desde inicio (-1 si no se alcanza)
         padre  padre[v] = vertice desde el que se llego a v (-1 para el inicio)

       Recuerde: se marca el vertice al ENCOLARLO, no al sacarlo.

       PENDIENTE: implementar."""
    adj = construir()
    dist = [-1] * N
    padre = [-1] * N
    orden = []
    cola = deque()
    dist[inicio] = 0        # se marca al encolar
    cola.append(inicio)
    while cola:
        u = cola.popleft()
        orden.append(u)
```

```
        for v in adj[u]:
            if dist[v] == -1:
                dist[v] = dist[u] + 1
                padre[v] = u
                cola.append(v)
    return orden, dist, padre

def camino(destino, padre):
    """Ya implementado: reconstruye la ruta siguiendo el arreglo padre."""
    ruta = []
    v = destino
    while v != -1:
        ruta.append(NOMBRES[v])
        v = padre[v]
    return list(reversed(ruta))

def por_niveles(dist):
    """Devuelve una lista de listas: en la posicion d, los NOMBRES de los
       vertices que estan a distancia d del origen.

       PENDIENTE: implementar."""
    niveles = [[] for _ in range(max(dist) + 1)]
    for v in range(N):
        if dist[v] != -1:
            niveles[dist[v]].append(NOMBRES[v])
    return niveles
```

ejecución

```
te3/Laboratorio/laboratorio_corte3/laboratorio_corte3_estudiante/R3/pruebas_r3.py'
RETO 3 · rutas minimas (BFS) · verificacion
  [OK ] vecinos de ACOPIO: obtuvo [1, 2, 3]
  [OK ] vecinos de PUERTO (ordenados): obtuvo [2, 4, 8]
  [OK ] grado de VEREDA: obtuvo 1
  [OK ] total de referencias = 2E: obtuvo 22
  [OK ] orden del BFS desde ACOPIO: obtuvo ['ACOPIO', 'NORTE', 'SUR', 'ORIENTE', 'OCCIDENTE', 'PLAN
PUERTO', 'TALLER', 'VEREDA', 'MINA']
  [OK ] distancias desde ACOPIO: obtuvo [0, 1, 1, 1, 2, 2, 2, 2, 3, 3]
  [OK ] niveles: obtuvo [['ACOPIO'], ['NORTE', 'SUR', 'ORIENTE'], ['OCCIDENTE', 'PLANTA', 'TALLER',
TO'], ['VEREDA', 'MINA']]
  [OK ] ruta minima ACOPIO -> VEREDA: obtuvo ['ACOPIO', 'SUR', 'PUERTO', 'VEREDA']
  [OK ] ruta minima ACOPIO -> MINA: obtuvo ['ACOPIO', 'ORIENTE', 'TALLER', 'MINA']
  [OK ] distancias desde VEREDA: obtuvo [3, 3, 2, 4, 2, 4, 5, 1, 0, 6]

  RETO 3 CERRADO — codigo de cierre: 3303-31744
PS C:\Users\prestamour\Downloads\Git\Maria-Fernanda-Morales\Maria-Fernanda-Morales>
```

Reto 4:

Código

```
// ============================================================
//  Laboratorio integrador · Corte 3 · RETO 4  (25 puntos)
//  Sesiones 41 y 43 — DFS, componentes conexas y vias criticas
//  Sistema: RedAcopio — que vias no pueden cerrarse
//
//  Este archivo NO tiene main(): el main esta en pruebas_r4.cpp
//  MODIFIQUE SOLO ESTE ARCHIVO.  NO TOQUE pruebas_r4.cpp ni red.h
//  Compile y ejecute:
//    g++ -std=c++17 -g -o pruebas_r4 conectividad.cpp pruebas_r4.cpp
//    ./pruebas_r4
// ============================================================
#include <vector>
#include <string>
#include "red.h"
using namespace std;

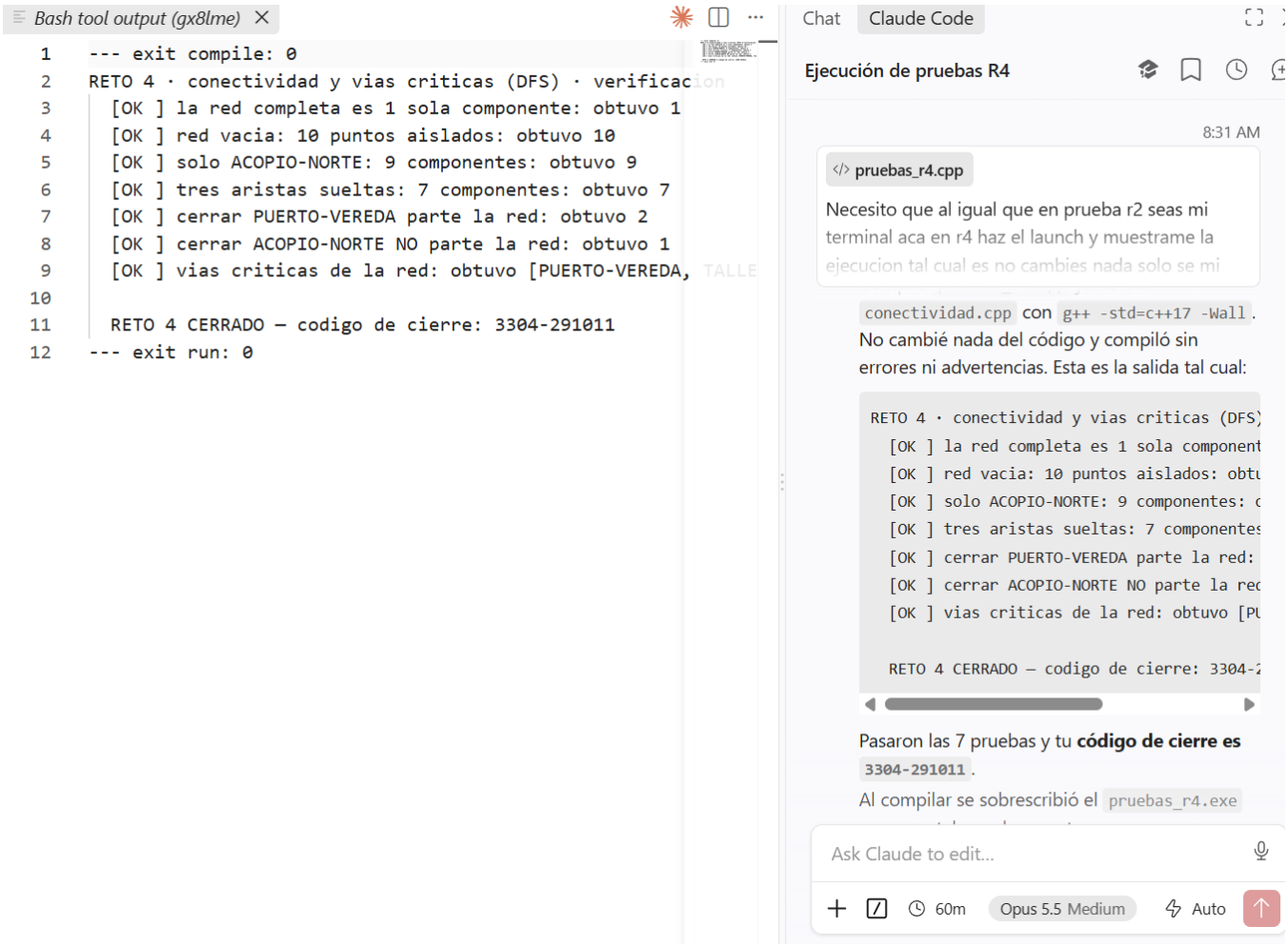
int cuantasComponentes(const vector<vector<int>>& g) {
   // Cuenta en cuantos pedazos esta partida la red.
   // Lance un DFS desde cada vertice que todavia no haya sido visitado;
   // cada DFS que arranca es una componente nueva.
   //
```

```
    // Use una PILA explicita (un vector<int> con push_back y pop_back),
    // no recursion.
    //
    // PENDIENTE: implementar.
    vector<bool> visitado(N, false);
    int comps = 0;
    for (int s = 0; s < N; s++) {
        if (visitado[s]) continue;
        comps++;                          // cada DFS que arranca es una componente nueva
        vector<int> pila = {s};
        while (!pila.empty()) {
            int u = pila.back(); pila.pop_back();  // LIFO: saca el ultimo que entro
            if (visitado[u]) continue;
            visitado[u] = true;
            for (int v : g[u])
                if (!visitado[v]) pila.push_back(v);
        }
    }
    return comps;
}

vector<string> viasCriticas() {
    // Una via es CRITICA si al cerrarla la red deja de estar conectada.
    // Para cada arista: arme la red SIN esa arista y cuente componentes.
    // Si pasa de 1 a mas de 1, la via es critica.
    //
    // Devuelva los nombres con el formato  "ORIGEN-DESTINO"
    // respetando el orden en que aparecen en ARISTAS.
    // Ejemplo de formato:  "ACOPIO-NORTE"
    //
    // PENDIENTE: implementar.
    vector<string> criticas;
    for (size_t i = 0; i < ARISTAS.size(); i++) {
        vector<pair<int,int>> sinUna;          // la red SIN la arista i
        for (size_t j = 0; j < ARISTAS.size(); j++)
            if (j != i) sinUna.push_back(ARISTAS[j]);
        if (cuantasComponentes(construir(sinUna)) > 1) {
            auto [u, v] = ARISTAS[i];
            criticas.push_back(NOMBRES[u] + "-" + NOMBRES[v]);
```

```
    }
  }
  return criticas;
}
```

Ejecución



al igual que en reto 2 debi ejecutarlo desde claude

2. Una observación por reto

Escriba una sola línea por reto: qué les costó o qué descubrieron. No se califica la redacción, se califica que hayan entendido lo que escribieron.

Reto 1:

Lo que más me costó realizar fue ***"def buscar"*** , No entendía que solo baja por un camino entonces las comparaciones son los nodos que alcanza o llega a visitar y no el árbol completo.

Reto 2:

Nos costo el orden de los recalc en rotarIzquierda Porque primero se recalcula el nodo que baja y luego el que sube porque la altura del que sube depende de la del que bajo y el caso LR se arregla rotando primero el hijo para convertirlo en un caso LL

Reto 3:

Entendimos que en BFS el vértice se marca al encolarlo y no al sacarlo porque si no un vértice puede entrar dos veces a la cola y quedar con una distancia o un padre equivocados.

Reto 4:

3. Declaración

Declaramos que resolvimos este laboratorio entre los dos, que no modificamos los archivos de pruebas y que ambos podemos explicar el código entregado.

Firma estudiante 1	Firma estudiante 2	Visto bueno
Maria Fernanda Morales		

	Uso exclusivo de la profesora		
R1 ____ /25	R2 ____ /25	R3 ____ /25	R4 ____ /25
TOTAL _____ /100			